

Run the Ford-Fulkerson algorithm on the flow network in Figure 24.8(c) and show the residual network after each flow augmentation. Number the vertices in L top to bottom from 1 to 5 and in R top to bottom from 6 to 9. For each iteration, pick the augmenting path that is lexicographically smallest.

24.3-2

Prove Theorem 24.10. Use induction on the number of iterations of the Ford-Fulkerson method.

24.3-3

Let $G = (V, E)$ be a bipartite graph with vertex partition $V = L \cup R$, and let G' be its corresponding flow network. Give a good upper bound on the length of any augmenting path found in G' during the execution of FORD-FULKERSON.

Problems

24-1 *Escape problem*

An $n \times n$ **grid** is an undirected graph consisting of n rows and n columns of vertices, as shown in Figure 24.9. We denote the vertex in the i th row and the j th column by (i, j) . All vertices in a grid have exactly four neighbors, except for the boundary vertices, which are the points (i, j) for which $i = 1$, $i = n$, $j = 1$, or $j = n$.

Given $m \leq n^2$ starting points $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$ in the grid, the **escape problem** is to determine whether there are m vertex-disjoint paths from the starting points to any m different points on the boundary. For example, the grid in Figure 24.9(a) has an escape, but the grid in Figure 24.9(b) does not.

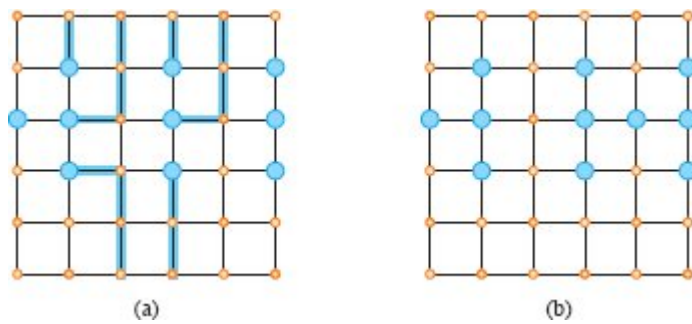


Figure 24.9 Grids for the escape problem. Starting points are blue, and other grid vertices are tan. **(a)** A grid with an escape, shown by blue paths. **(b)** A grid with no escape.

- a.** Consider a flow network in which vertices, as well as edges, have capacities. That is, the total positive flow entering any given vertex is subject to a capacity constraint. Show how to reduce the problem of determining the maximum flow in a network with edge and vertex capacities to an ordinary maximum-flow problem on a flow network of comparable size.
- b.** Describe an efficient algorithm to solve the escape problem, and analyze its running time.

24-2 Minimum path cover

A **path cover** of a directed graph $G = (V, E)$ is a set P of vertex-disjoint paths such that every vertex in V is included in exactly one path in P . Paths may start and end anywhere, and they may be of any length, including 0. A **minimum path cover** of G is a path cover containing the fewest possible paths.

- a.** Give an efficient algorithm to find a minimum path cover of a directed acyclic graph $G = (V, E)$. (*Hint:* Assuming that $V = \{1, 2, \dots, n\}$, construct a flow network based on the graph $G' = (V', E')$, where

$$V' = \{x_0, x_1, \dots, x_n\} \cup \{y_0, y_1, \dots, y_n\},$$

$$E' = \{(x_0, x_i) : i \in V\} \cup \{(y_i, y_0) : i \in V\} \cup \{(x_i, y_j) : (i, j) \in E\},$$

and run a maximum-flow algorithm.)

- b.** Does your algorithm work for directed graphs that contain cycles? Explain.

24-3 *Hiring consulting experts*

Professor Fieri wants to open a consulting company for the food industry. He has identified n important food categories, which he represents by the set $C = \{C_1, C_2, \dots, C_n\}$. In each category C_k , he can hire an expert in that category for $e_k > 0$ dollars. The consulting company has lined up a set $J = \{J_1, J_2, \dots, J_m\}$ of potential jobs. In order to perform job J_i , the company needs to have hired experts in a subset $R_i \subseteq C$ of categories. Each expert can work on multiple jobs simultaneously. If the company chooses to accept job J_i , it must have hired experts in all categories in R_i , and it takes in revenue of $p_i > 0$ dollars.

Professor Fieri's job is to determine which categories to hire experts in and which jobs to accept in order to maximize the net revenue, which is the total income from jobs accepted minus the total cost of employing the experts.

Consider the following flow network G . It contains a source vertex s , vertices $C_1, C_2, \dots, C_n$, vertices $J_1, J_2, \dots, J_m$, and a sink vertex t . For $k = 1, 2, \dots, n$, the flow network contains an edge (s, C_k) with capacity $c(s, C_k) = e_k$, and for $i = 1, 2, \dots, m$, the flow network contains an edge (J_i, t) with capacity $c(J_i, t) = p_i$. For $k = 1, 2, \dots, n$ and $i = 1, 2, \dots, m$, if $C_k \in R_i$, then G contains an edge (C_k, J_i) with capacity $c(C_k, J_i) = \infty$.

- a.** Show that if $J_i \in T$ for a finite-capacity cut (S, T) of G , then $C_k \in T$ for each $C_k \in R_i$.
- b.** Show how to determine the maximum net revenue from the capacity of a minimum cut of G and the given p_i values.
- c.** Give an efficient algorithm to determine which jobs to accept and which experts to hire. Analyze the running time of your algorithm in terms of m , n , and $r = \sum_{i=1}^m |R_i|$.

24-4 *Updating maximum flow*

Let $G = (V, E)$ be a flow network with source s , sink t , and integer capacities. Suppose that you are given a maximum flow in G .

- a. Suppose that the capacity of a single edge $(u, v) \in E$ increases by 1. Give an $O(V + E)$ -time algorithm to update the maximum flow.
- b. Suppose that the capacity of a single edge $(u, v) \in E$ decreases by 1. Give an $O(V + E)$ -time algorithm to update the maximum flow.

24-5 *Maximum flow by scaling*

Let $G = (V, E)$ be a flow network with source s , sink t , and an integer capacity $c(u, v)$ on each edge $(u, v) \in E$. Let $C = \max \{c(u, v) : (u, v) \in E\}$.

- a. Argue that a minimum cut of G has capacity at most $C|E|$.
- b. For a given number K , show how to find an augmenting path of capacity at least K in $O(E)$ time, if such a path exists.

The procedure MAX-FLOW-BY-SCALING appearing on the following page modifies the basic FORD-FULKERSON-METHOD procedure to compute a maximum flow in G .

- c. Argue that MAX-FLOW-BY-SCALING returns a maximum flow.
- d. Show that the capacity of a minimum cut of the residual network G_f is less than $2K|E|$ each time line 4 executes.
- e. Argue that the inner **while** loop of lines 5–6 executes $O(E)$ times for each value of K .

MAX-FLOW-BY-SCALING (G, s, t)

```
1  $C = \max \{c(u, v) : (u, v) \in E\}$ 
2 initialize flow  $f$  to 0
3  $K = 2^{\lfloor \lg C \rfloor}$ 
4 while  $K \geq 1$ 
5   while there exists an augmenting path  $p$  of capacity at least  $K$ 
```

```

6      augment flow  $f$  along  $p$ 
7       $K = K/2$ 
8  return  $f$ 

```

f. Conclude that MAX-FLOW-BY-SCALING can be implemented so that it runs in $O(E^2 \lg C)$ time.

24-6 Widest augmenting path

The Edmonds-Karp algorithm implements the Ford-Fulkerson algorithm by always choosing a shortest augmenting path in the residual network. Suppose instead that the Ford-Fulkerson algorithm chooses a *widest augmenting path*: an augmenting path with the greatest residual capacity. Assume that $G = (V, E)$ is a flow network with source s and sink t , that all capacities are integer, and that the largest capacity is C . In this problem, you will show that choosing a widest augmenting path results in at most $|E| \ln |f^*|$ augmentations to find a maximum flow f^* .

- a.* Show how to adjust Dijkstra's algorithm to find the widest augmenting path in the residual network.
- b.* Show that a maximum flow in G can be formed by successive flow augmentations along at most $|E|$ paths from s to t .
- c.* Given a flow f , argue that the residual network G_f has an augmenting path p with residual capacity $c_f(p) \geq (|f^*| - |f|)/|E|$.
- d.* Assuming that each augmenting path is a widest augmenting path, let f_i be the flow after augmenting the flow by the i th augmenting path, where f_0 has $f(u, v) = 0$ for all edges (u, v) . Show that $|f^*| - |f_i| \leq |f^*| (1 - 1/|E|)^i$.
- e.* Show that $|f^*| - |f_i| < |f^*| e^{-i/|E|}$.
- f.* Conclude that after the flow is augmented at most $|E| \ln |f^*|$ times, the flow is a maximum flow.

24-7 Global minimum cut

A **global cut** in an undirected graph $G = (V, E)$ is a partition (see page 1156) of V into two nonempty sets V_1 and V_2 . This definition is like the definition of cut that we have used in this chapter, except that we no longer have distinguished vertices s and t . Any edge (u, v) with $u \in V_1$ and $v \in V_2$ is said to **cross** the cut.

We can extend this definition of a cut to a multigraph $G = (V, E)$ (see page 1167), and we denote by $c(u, v)$ the number of edges in the multigraph with endpoints u and v . A global cut in a multigraph is still a partition of the vertices, and the value of a global cut (V_1, V_2) is $c(V_1, V_2) = \sum_{u \in V_1, v \in V_2} c(u, v)$. A solution to the **global-minimum-cut problem** is a cut (V_1, V_2) such that $c(V_1, V_2)$ is minimum. Let $\mu(G)$ denote the value of a global minimum cut in a graph or multigraph G .

- a. Show how to find a global minimum cut of a graph $G = (V, E)$ by solving $\binom{|V|}{2}$ maximum-flow problems, each with a different pair of vertices as the source and sink, and taking the minimum value of the cuts found.
- b. Give an algorithm to find a global minimum cut by solving only $\Theta(V)$ maximum-flow problems. What is the running time of your algorithm?

The remainder of this problem develops an algorithm for the global-minimum-cut problem that does not use any maximum-flow computations. It uses the notion of an edge contraction, defined on page 1168, with one crucial difference. The algorithm maintains a multigraph, so that upon contracting an edge (u, v) , it creates a new vertex x , and for any other vertex $y \in V$, the number of edges between x and y is $c(u, y) + c(v, y)$. The algorithm does not maintain self-loops, and so it sets $c(x, x)$ to 0. Denote by $G/(u, v)$ the multigraph that results from contracting edge (u, v) in multigraph G .

Consider what can happen to the minimum cut when an edge is contracted. Assume that, at all points, the minimum cut in a multigraph G is unique. We'll remove this assumption later.

- c. Show that for any edge (u, v) , we have $\mu(G/(u, v)) \leq \mu(G)$. Under what conditions is $\mu(G/(u, v)) < \mu(G)$?

Next, you will show that if you pick an edge uniformly at random, the probability that it belongs to the minimum cut is small.

- d. Show that for any multigraph $G = (V, E)$, the value of the global minimum cut is at most the average degree of a vertex: that $\mu(G) \leq 2|E|/|V|$, where $|E|$ denotes the total number of edges in the multigraph.
- e. Using the results from parts (c) and (d), show that, if we pick an edge (u, v) uniformly at random, then the probability that (u, v) belongs to the minimum cut is at most $2/V$.

Consider the algorithm that repeatedly chooses an edge at random and contracts it until the multigraph has exactly two vertices, say u and v . At that point, the multigraph corresponds to a cut in the original graph, with vertex u representing all the nodes in one side of the original graph, and v representing all the vertices on the other side. The number of edges given by $c(u, v)$ corresponds exactly to the number of edges crossing the corresponding cut in the original graph. We call this algorithm the *contraction algorithm*.

- f. Suppose that the contraction algorithm terminates with a multigraph whose only vertices are u and v . Show that $\Pr\{c(u, v) = \mu(G)\} = \Omega\left(1/\binom{|V|}{2}\right)$.
- g. Prove that if the contraction algorithm repeats $\binom{|V|}{2} \ln |V|$ times, then the probability that at least one of the runs returns the minimum cut is at least $1 - 1/|V|$.
- h. Give a detailed implementation of the contraction algorithm that runs in $O(V^2)$ time.
- i. Combine the previous parts and remove the assumption that the minimum cut must be unique, to conclude that running the contraction algorithm $\binom{|V|}{2} \ln |V|$ times yields an algorithm that runs in $O(V^4 \lg V)$ time and returns a minimum cut with probability at least $1 - 1/V$.

Chapter notes

Ahuja, Magnanti, and Orlin [7], Even [137], Lawler [276], Papadimitriou and Steiglitz [353], Tarjan [429], and Williamson [458] are good references for network flows and related algorithms. Schrijver [399] has written an interesting review of historical developments in the field of network flows.

The Ford-Fulkerson method is due to Ford and Fulkerson [149], who originated the formal study of many of the problems in the area of network flow, including the maximum-flow and bipartite-matching problems. Many early implementations of the Ford-Fulkerson method found augmenting paths using breadth-first search. Edmonds and Karp [132], and independently Dinic [119], proved that this strategy yields a polynomial-time algorithm. A related idea, that of using “blocking flows,” was also first developed by Dinic [119].

A class of algorithms known as *push-relabel algorithms*, due to Goldberg [185] and Goldberg and Tarjan [188], takes a different approach from the Ford-Fulkerson method. Push-relabel algorithms allow flow conservation to be violated at vertices other than the source and sink as they execute. Using an idea first developed by Karzonov [251], they allow a *preflow* in which the flow into a vertex may exceed the flow out of the vertex. Such a vertex is said to be *overflowing*. Initially, every edge leaving the source is filled to capacity, so that all neighbors of the source are overflowing. In a push-relabel algorithm, each vertex is assigned an integer height. An overflowing vertex may push flow to a neighboring vertex to which it has a residual edge provided that it is higher than the neighbor. If all residual edges from an overflowing vertex go to neighbors with equal or greater heights, then the vertex may increase its height. Once all vertices other than the sink are no longer overflowing, the preflow is not only a legal flow, but also a maximum flow.

Goldberg and Tarjan [188] gave an $O(V^3)$ -time algorithm that uses a queue to maintain the set of overflowing vertices, as well as an algorithm that uses dynamic trees to achieve a running time of $O(VE \lg(V^2/E + 2))$. Several other researchers developed improved variants

and implementations [9, 10, 15, 86, 87, 255, 358], the fastest of which, by King, Rao, and Tarjan [255], runs in $O(VE \log E / (V \lg V) \lg V)$ time.

Another efficient algorithm for maximum flow, by Goldberg and Rao [187], runs in $O(\min\{V^{2/3}, E^{1/2}\} E \lg(V^2/E + 2) \lg C)$ time, where C is the maximum capacity any edge. Orlin [350] gave an algorithm in the same spirit as this algorithm that runs in $O(VE + E^{31/16} \lg^2 V)$ time. Combining it with the algorithm of King, Rao, and Tarjan results in an $O(VE)$ -time algorithm.

A different approach to maximum flows and related problems is to use techniques from continuous optimization including electrical flows and interior-point methods. The first breakthrough in this line of work is due to Madry [308], who gave an $\tilde{O}(E^{10/7})$ -time algorithm for unit-capacity maximum flow and bipartite maximum matching. (See Problem 3-6 on page 73 for a definition of $\tilde{O}$.) There has been a series of papers in this area for matchings, maximum flows, and minimum-cost flows. The fastest algorithm to date in this line of work for maximum flow is due to Lee and Sidford [285], taking $\tilde{O}(\sqrt{VE} \lg^{o(1)} C)$ time. If the capacities are not too large, this algorithm is faster than the $O(VE)$ -time algorithm mentioned above. Another algorithm, due to Liu and Sidford [303] runs in $\tilde{O}(E^{11/8} C^{1/4})$ time, where C is the maximum capacity of any edge. This algorithm does not run in polynomial time, but for small enough capacities, it is faster than the previous ones.

In practice, push-relabel algorithms currently dominate algorithms based on augmenting paths, continuous-optimization, and linear programming for the maximum-flow problem [88].

¹ Recall from Section 20.1 that we represent an attribute f for edge (u, v) with the same style of notation— $(u, v).f$ —that we use for an attribute of any other object.